

第1章 拡張子は信用できない

この章で学ぶこと

- 拡張子は信用できない
- ファイルにはシグネチャがある
- ハッシュによる改ざん検知

拡張子とは

拡張子のことはご存知でしょうか？

拡張子とはファイルの形式を示す文字列の名称のことです。

例えば、pdfやzip、png、xlsxなどさまざまな種類の拡張子があります。

そこで、次の2つのpdfを用意しました。

今日	▼	サイズ	種類
 ao.pdf		56 KB	PDF 書類
 aka.pdf		132.6 MB	PDF 書類

どちらも本当にpdfでしょうか？

これを確かめるためのコマンドに `file` というコマンドがあります。

```
~/Downloads master
) file ao.pdf
ao.pdf: PDF document, version 1.3, 1 pages

~/Downloads master
) file aka.pdf
aka.pdf: ASCII text, with very long lines (8439)
```

file <対象のファイル名>

上記の形式でターミナルに打ち込むことによって、
対象のファイルがどんなファイルかを解析することができます。

今回の場合、ao.pdfは本当にpdfで、aka.pdfは偽造されたpdf（txtファイル）でした。
aka.pdfを**aka.txt**に変えることで実際に開くことができます。



また、`file` にはさまざまな**オプション**があります。

`-i` オプション：**MIMEタイプ**を表示

例：`file -i a.txt`

出力例：`a.txt: text/plain; charset=us-ascii`

▼ MIMEタイプとは

MIMEタイプとは「**タイプ名/サブタイプ名**」の形式の文字列で、WEBサーバーとWEBブラウザの間はこのMIMEタイプを用いてデータの形式を指定しています。

例：

- 1.txtの場合はMIMEタイプはtext/plain
- 2.pdfの場合はMIMEタイプはapplication/pdf

`file` の応用例として、次のような使い方ができます。

`file *` と入力し、実行すると、実行したディレクトリ内の**すべてのファイルの種類**を確認することができます。

```
14_lsr:      directory
a.class:     compiled Java class data, version 68.0
a.java:      c program text, Unicode text, UTF-8 text
a.out:       Mach-O 64-bit executable arm64
a.out.dSYM:  directory
algo.py:     Python script text executable, ASCII text
AndroidStudioProjects: directory
api-arch.drawio: exported SGML document text, Unicode text, UTF-8 text
Applications: directory
```

fileの仕組み

なぜ拡張子がpdfなのに `file` を実行するとtxtと出たのでしょうか？

`file` の簡単な流れ：

- 1: ファイル先頭付近の**バイト列**を読む

2：既知のシグネチャ（パターン）と比較

3：一致した形式を返す

▼ バイト列とは

複数の「バイト（8個の0か1が並んだもの）」が順序を持って並んだデータの集まりのことです。

文字、画像、音声、プログラムなどコンピュータに関することはほぼ全てバイト列でできています。

以上の流れで `file` はファイル形式を判定しています。

例えば、先ほどあったao.pdfのファイル先頭付近のバイト列を読んで、シグネチャと比較してみましょう。

https://en.wikipedia.org/wiki/List_of_file_signatures

以上のリストを参考にする。

character where available, or a box character wise. In some cases the space character is shown

Hex signature	ISO 8859-1	Offset	Extension
25 50 44 46 2D	%PDF-	0	pdf

これがPDFのシグネチャのバイト列です。Offsetは先頭からの位置、Hex signatureがバイト列の並びとそれぞれの値です。

```
~/Downloads master
> xxd ao.pdf | head
00000000: 2550 4446 2d31 2e33 0a25 c4e5 f2e5 eba7 %PDF-1.3.%.....
00000010: f3a0 d0c4 c60a 3320 3020 6f62 6a0a 3c3c .....3 0 obj.<<
00000020: 202f 4669 6c74 6572 202f 466c 6174 6544 /Filter /FlateD
00000030: 6563 6f64 6520 2f4c 656e 6774 6820 3833 ecode /Length 83
00000040: 203e 3e0a 7374 7265 616d 0a78 012b 5408 >>.stream.x.+T.
00000050: 5428 5430 0042 534b 5305 0b13 2385 a254 T(T0.BSKS...#..T
00000060: 8570 853c 05fd 80d4 a2e4 d482 92d2 c41c .p.<.....
00000070: 85a2 4cb0 1a0b 634b 3d33 1303 2305 5d90 ..L...cK=3..#..].
00000080: 5a88 0e43 3d43 734b 0b0b 6385 e45c 2e7d Z..C=CsK..c..\}.
00000090: cf5c 4305 977c a089 8100 b7e9 151a 0a65 .\C..|.....e
```

▼ コマンド解説

xxdとは、対象ファイルのバイト列（16進数）を表示するコマンドです。

| はパイプというもので、前のコマンドの出力結果を次のコマンドへ渡す役割があります。

headは、オプションなしの場合、先頭から10行だけを表示するものです。

つまり、画像のコマンドは**ファイルを16進数のバイト列で表示し、その先頭10行だけを見る**という意味になります。

ファイル先頭付近のバイト列を見ると、しっかりPDFの**ファイルシグネチャ**を確認できました。

なぜfileは重要か

悪意のあるファイルには次のようにファイルの拡張子を偽造するものがあります。代表的なものとして以下の3つがあります。

- 1：参考資料.pdf.exeのように**二重拡張子**のもの
- 2：参考資料{大量のスペース}.exeのように**大量のスペース**による偽造のもの
- 3：参考資料{RLO}fdp.exe（参考資料exe.pdfに見える）のように**RLO**という技術を悪用したもの

▼ RLOとは

Right-to-Left Overrideの略で、文字列の表示順序を右から左に変えるもの。例えばfdp.exeが元々の形式の場合はexe.pdfのように見えます。

exeは**Windows**での**実行ファイル**です。

悪意のある**exe**ファイルをクリックしてしまうと、**マルウェア**に感染してしまう恐れがあります。

以上のように拡張子を偽造したものでも **file** を使えば見抜くことが可能です。

外部のアプリまたはファイルをダウンロードした時、拡張子がおかしいと感じた場合はぜひ **file** を試してみてください。

ハッシュ

コンピュータの世界では、**ハッシュ値**というものを見ることがよくあります。

例えば、アプリのダウンロードリンクの下側などに

SHA-256：abcd...xyz（ハッシュ値）のように置かれていることがあります。

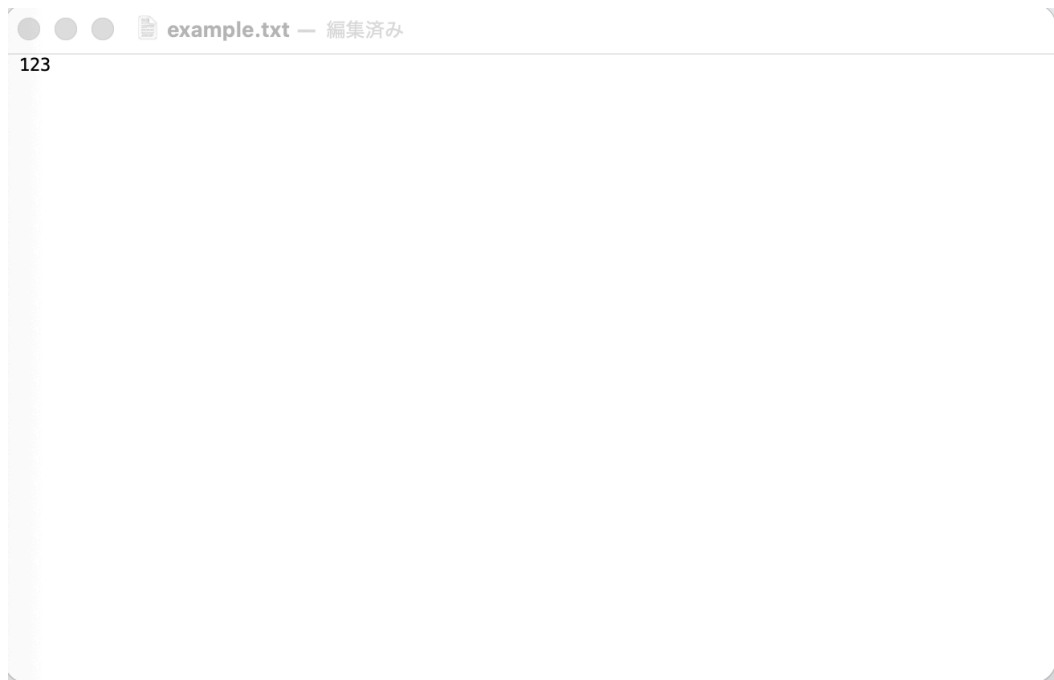
▼ SHA-256とは

SHA-256とは任意の長さの原文から固定長の特徴的な値を算出するハッシュ関数の一つです。

ハッシュ値が置かれている理由は**改ざんを検知するため**です。

どのように改ざんを検知しているのかを見ていきましょう。

123という内容が書かれたtxtファイルを用意しました。



SHA-256というハッシュアルゴリズムを用いたハッシュ値を出力するコマンドを実行すると、次のように表示されます。

```
~/Downloads master  
$ shasum -a 256 example.txt  
a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e998e86f7f7a27ae3 example.txt
```

▼ コマンド解説

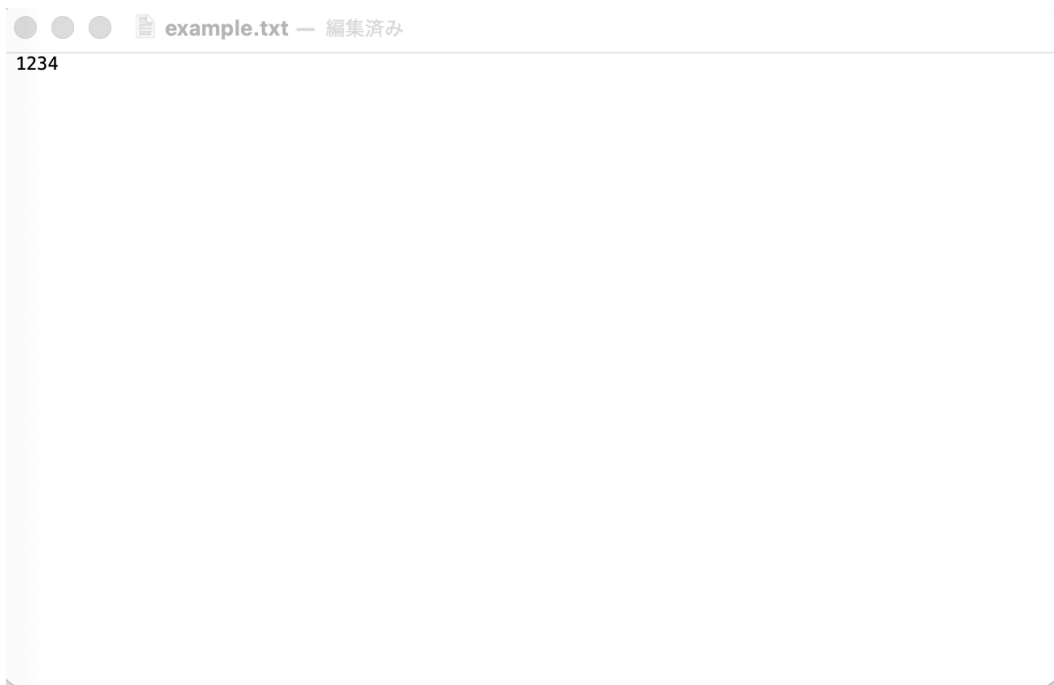
```
shasum -a 256 example.txt
```

shasum : `shasum <対象のファイル>` で対象ファイルのハッシュ値を出力させることができる

オプション a : 使用するSHAアルゴリズムを指定

今回はSHA-256

次に、123と書かれたtxtファイルに対して、4を追記しました。



同じようにハッシュ値を出力するコマンドを実行すると、次のように表示されます。

```
~/Downloads master  
$ shasum -a 256 example.txt  
03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4 example.txt
```

123の場合：

a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e998e86f7f7a27ae3

1234の場合：

03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4

同じような内容でも、1文字でも変われば、全然違うハッシュ値になるということを確認できました。

よって、もし配布元が公開しているハッシュ値と、自分がダウンロードしたファイルのハッシュ値が一致していれば、改ざんされていない可能性が高いと言えます。

将来マイナーなソフトウェアをダウンロードした時に、ハッシュ値があれば確認してみてください。**命拾いすることがあるかもしれません。**

また、ハッシュ値はマルウェア対策にもよく利用されています。

例：

- 1 怪しいファイルのハッシュ値を確認
- 2 過去に発見されたマルウェアのハッシュ値と照合する。
- 3 一致すれば即座にマルウェアと判断することができます。

めんどくさい解析なしでマルウェアを発見することができるので、有用な手段だと言えます。

まとめ

`file` とはファイルの種類を判定するコマンドです。

`file <ファイル名>` という形で使用されます。

拡張子ではなくファイル先頭のバイト列（ファイルシグネチャ）を読んで判定しています。

悪意のあるファイルは拡張子を偽造してくることがあり、`file` ではそういった偽造も見抜けるので重要です。

ハッシュ値は改ざん検知に活用されており、マルウェア対策にも活用されています。

`shasum -a 256 <ファイル名>` で対象ファイルをSHA-256アルゴリズムを用いたハッシュ値を出力する。

以下の課題に取り組んでみてください。

問題1：ディレクトリ1にあるcat.pngとdog.pdf、mystery.zip、kindai.heicに関して、実際はそれぞれ何のファイルですか？

（例：Aはpdf、Bはpng...）

問題2：ディレクトリ1にあるsample1とsample2、sample1とsample3に関して、それぞれ同じファイルですか？異なるファイルですか？理由も教えてください。

（例：sample1とsample2は同じファイルと考えられます。なぜなら～だからです。）

参考資料

<https://engineers.ffri.jp/entry/2021/12/09/121107>

<https://qiita.com/free-honda/items/6dd53f687eda24c8f77e>